GUIDE Détaillé : XPath,XSLT, JSON

1. Qu'est-ce que XPATH

Un document XML est constitué d'une hiérarchie d'éléments. Une expression XPath permet de naviguer à travers les éléments et les attributs d'un document XML.

1.1. Exemples XPath

Étant donné le fichier XML suivant :

```
<topic id="abc">
  <title>Using XPATH</title>
  <body>
    <p>Using XPATH is easy.</p>
    <fig>
      <image href="images/xpath.png"/>
    </fig>
    <section>
      <title>Examples</title>
      <p audience="novice">A simple example.</p>
      <p audience="expert">An advanced example.</p>
      <p audience="expert">Another advanced example.</p>
      <fig>
        <image href="images/xpath-axes.png">
          <alt>This screenshot shows the XPATH axes</alt>
        </image>
      </fig>
    </section>
    <p>The End.</p>
  </body>
</topic>
```

Le tableau suivant contient une liste de quelques expressions XPath pour l'exemple XML donné ci-dessus.

Requête XPath	**Localise**
/topic	Renvoie l' <topic>élément racine.
//title	Renvoie n'importe quel <title>élément.
//section/title	Renvoie uniquement l' <title>élément enfant d'un <section>élément.
//p	Renvoie n'importe quel <p>élément.
//p[@audience='expert']	Renvoie tout <p>élément dont l' @audienceattribut est défini sur expert.
//p[not(@audience)]	Renvoie tout <p>élément pour lequel l' @audienceattribut est manquant.
//p[not(@audience='admin')]	Renvoie tout <p>élément dont l' @audienceattribut n'a pas de valeur adminOU est manquant.
//p[text()='To start this process']	Renvoie tous <p>les éléments qui commencent par la chaîne de texte To start this process.
//p[contains(.,'button')]	Renvoie tout <p>élément contenant la chaîne de caractères buttonquelque part dans le texte.
//image[not(alt)]	Renvoie tout <image>élément auquel il manque un <alt>élément enfant.

- XPath peut être exécuté dans le Navigateur web (HTML) utilisant $x ()

Par exemple : $x ("//h1").

2. Qu'est-ce que XSLT

XSLT (eXtensible Stylesheet Language Transformations) est un langage utilisé pour transformer des documents XML en :

- HTML (pages web)
- XML (autre structure)
- Texte simple

2.1. Structure de base d'un fichier XSLT

Un fichier XSLT est lui-même un document XML.

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <!-- Contenu ici -->
  </xsl:template>
</xsl:stylesheet>
```

Explication :

- <xsl:stylesheet> : racine du document XSLT
- <xsl:template> : règle de transformation
- match="/" : signifie "appliquer à la racine du document XML"

2.2. Exemple simple

XML (source)

```
<etudiants>
  <etudiant>
    <nom>Ali</nom>
    <age>20</age>
  </etudiant>
  <etudiant>
    <nom>Sara</nom>
```

```
    <age>22</age>
  </etudiant>
</etudiants>
```

XSLT (transformation vers HTML)

```
<xsl:template match="/">
  <html>
    <body>
      <h2>Liste des étudiants</h2>
      <ul>
        <xsl:for-each select="etudiants/etudiant">
          <li>
            <xsl:value-of select="nom"/> -
            <xsl:value-of select="age"/> ans
          </li>
        </xsl:for-each>
      </ul>
    </body>
  </html>
</xsl:template>
```

Résultat HTML

- Ali - 20 ans

- Sara - 22 ans

2.3. Instructions importantes

2.3.1. xsl:value-of

- Permet d'afficher une valeur

```
<xsl:value-of select="nom"/>
```

2.3.2. xsl:for-each

- Permet de parcourir une liste

```
<xsl:for-each select="etudiants/etudiant">
```

2.3.3. xsl:if

- Condition simple

```
<xsl:if test="age > 21">
   <p>Étudiant majeur</p>
</xsl:if>
```

2.3.4. xsl:choose (équivalent de if/else)

```
<xsl:choose>
   <xsl:when test="age &lt; 18">
      Mineur
   </xsl:when>
   <xsl:otherwise>
      Majeur
   </xsl:otherwise>
</xsl:choose>
```

2.3.5. xsl:apply-templates

- Permet d'appliquer des règles automatiquement

```
<xsl:apply-templates select="etudiants/etudiant"/>
```

Et ailleurs :

```
<xsl:template match="etudiant">
   <p><xsl:value-of select="nom"/></p>
</xsl:template>
```

3. XPath (très important)

XSLT utilise **XPath** pour naviguer dans XML.

Exemples :

Expression	Signification
/	Racine
etudiants/etudiant	Tous les étudiants
nom	Élément nom
@id	Attribut id

4. Trier les données

```
<xsl:for-each select="etudiants/etudiant">
  <xsl:sort select="age" order="ascending"/>
```

- Trie les étudiants par âge

5. Résumé

- XSLT transforme XML
- XPath sert à naviguer
- Templates = règles
- For-each = boucle
- If / choose = conditions

6. Introduction to JSON

JSON (JavaScript Object Notation) est un format léger utilisé pour stocker et échanger des données entre systèmes (applications, serveurs, sites web, etc.). Il est facile à lire pour les humains et facile à traiter pour les machines.

6.1. Structure de base du JSON

Un fichier JSON est composé de paires clé-valeur :

```
{
  "nom": "Ali",
  "age": 16,
  "ville": "Oran"
}
```

6.2. Types de données en JSON

JSON supporte plusieurs types :

- Chaîne de caractères (string) → "bonjour"
- Nombre (number) → 25, 3.14
- Booléen (boolean) → true / false
- Tableau (array) → [1, 2, 3]
- Objet (object) → { "a": 1 }
- Null → null

6.2.1. Exemple plus complet

```
{
  "nom": "Sara",
  "age": 17,
  "estEtudiante": true,
  "notes": [15, 14, 18],
  "adresse": {
    "rue": "Rue 5",
    "ville": "Oran"
  }
}
```

6.3. À quoi sert JSON ?

- Échanger des données entre client et serveur (API)
- Stocker des configurations
- Utilisé dans les applications web et mobiles

7. JSON Schema Validation

JSON Schema Validation permet de vérifier si un fichier JSON respecte une structure définie (types, champs obligatoires, formats, etc.).

7.1. Qu'est-ce qu'un JSON Schema ?

C'est un fichier JSON qui décrit **les règles** que doit suivre un autre JSON.

Exemple simple de données :

```
{
  "title": "Le Petit Prince",
  "author": "Antoine de Saint-Exupéry",
  "year": 1943,
  "available": true,
  "keywords": ["enfant", "philosophie"]
}
```

7.2. Exemple de JSON Schema

Voici un schéma qui valide les données ci-dessus :

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "required": ["title", "author", "year"],
  "properties": {
    "title": {
      "type": "string"
    },
    "author": {
      "type": "string"
    },
    "year": {
      "type": "integer",
      "minimum": 0
    },
    "available": {
      "type": "boolean"
    },
    "keywords": {
      "type": "array",
      "items": {
        "type": "string"
      }
    }
  }
}
```

7.2.1. Explication

- "type": "object" → le JSON doit être un objet
- "required" → champs obligatoires
- "properties" → définit les champs autorisés
- "type" → type de chaque champ
- "minimum" → contrainte sur les nombres
- "items" → type des éléments d'un tableau

7.3. Exemple de validation

JSON valide :

```
{
  "title": "Livre",
  "author": "Auteur",
  "year": 2000,
  "available": false,
  "keywords": ["test"]
}
```

JSON invalide (year incorrect) :

```
{
  "title": "Livre",
  "author": "Auteur",
  "year": "2000"
}
```

- Erreur : "year" doit être un **nombre**, pas une chaîne.

7.4. Pourquoi utiliser JSON Schema ?

- Vérifier les données automatiquement
- Éviter les erreurs dans les API
- Documenter la structure des données
- Assurer la cohérence